GPU-Accelerated Covariance-Based Calibration for CHORD

DRAFT, living document. Last major restructure: 2026-11-01

Mike J. Sievers

November 1, 2026

Draft status. Current argument: CorrCal's covariance model calibrates quasi-redundant arrays where redundant calibration is biased, and a CUDA implementation makes it tractable at CHORD scale. Dead ends live in `attic/attic.tex`. Results log: `gpu_corrcal/results/results.md`.

Open items

- Write last. Placeholder sentences only. . . . 2
- Cite redundant cal papers and non-redundancy studies (Zotero tag: redcal). . . . 2
- check correct description of Pascua 2026 contribution 2
- Paragraph: why CHORD scale (antenna count, frequency channels, cadence) makes the CPU implementation the bottleneck. . . . 2
- Verify notation against Sievers 2017 and current repo docstrings. . . . 2
- Short subsection contrasting with redundant calibration: what assumption each makes, where each fails. This is also slide 3 of the Nov 9 group meeting talk. . 3
- Figure: memory layout diagram (group edges, block structure). Reuse for talk. . 3
- Draft from repo design notes `docs/kernels.md`. Keep to what a reader needs to reproduce performance; full design-pattern discussion goes to the thesis chapter. 3
- State why: FP32 accumulation failed, see attic A1. One sentence here, the detail stays in the attic/thesis. . . . 3
- Decide hardware to report (cluster GPU model) and fix the benchmark protocol before running final numbers. . . . 4
- Earlier version claimed CorrCal also improves convergence speed. Not supported, moved to attic A3. Do not reintroduce without new evidence. . . . 4
- Fill once results are final. Candidate points: limits of the low-rank diffuse model; application to real CHORD data; extensions (time and frequency correlated gains). . . . 5

Abstract

Write last. Placeholder sentences only.

We present a GPU implementation of CorrCal for the CHORD array. [Headline speedup]. [Headline calibration accuracy vs. redundant calibration].

1 Introduction

Claim: Precision 21 cm cosmology with CHORD requires calibration that tolerates array non-redundancy, and covariance-based calibration does this at a compute cost that only a GPU implementation makes practical.

Interferometric 21 cm experiments require per-antenna complex gains calibrated to high precision, since gain errors couple bright foregrounds into the cosmological signal. Redundant calibration exploits repeated baselines but assumes identical visibilities within each redundant group, an assumption violated by position errors and beam variations

Cite redundant cal papers and non-redundancy studies (Zotero tag: redcal).

CorrCal [2] replaces the exact-redundancy constraint with a statistical model of the sky covariance, and [1] extends [...].

check correct description of Pascua 2026 contribution

Paragraph: why CHORD scale (antenna count, frequency channels, cadence) makes the CPU implementation the bottleneck.

Contributions.

1. A CUDA implementation of the CorrCal likelihood and gradient.
2. Validation against the reference CPU code.

dundant calibration.

2 Background: covariance-based calibration

Claim: CorrCal models the visibility covariance as noise plus a low-rank sky term, so the likelihood can be evaluated with Woodbury identities instead of a dense inverse.

The measured visibilities are $\mathbf{d} = \mathbf{G}\,\mathbf{v} + \mathbf{n}$, with $\mathbf{G}$ the gain matrix. CorrCal models the data covariance as

$$\mathbf{C} = \mathbf{N} + \mathbf{G}\left(\mathbf{\Delta}\mathbf{\Delta}^\dagger + \mathbf{\Sigma}\mathbf{\Sigma}^\dagger\right)\mathbf{G}^\dagger, \tag{1}$$

where $\mathbf{N}$ is diagonal noise, $\mathbf{\Delta}$ holds low-rank diffuse-sky eigenmodes within each quasi-redundant group, and $\mathbf{\Sigma}$ holds the predicted responses to known point sources.

Verify notation against Sievers 2017 and current repo docstrings.

Gains are found by minimising

$$\chi^2(\mathbf{g}) = \mathbf{d}^\dagger \mathbf{C}^{-1}(\mathbf{g})\, \mathbf{d}, \tag{2}$$

with $\mathbf{C}^{-1}$ applied via the Woodbury identity, so the cost scales with the number of sky modes rather than the number of baselines squared.

Short subsection contrasting with redundant calibration: what assumption each makes, where each fails. This is also slide 3 of the Nov 9 group meeting talk.

3 GPU implementation

Claim: The likelihood and its gradient decompose into batched per-group dense operations plus sparse gain application, which map onto custom CUDA kernels with one thread block per redundant group.

3.1 Data layout

Visibilities are stored sorted by redundant group with an edge array marking group boundaries, so each group's diffuse-sky block is contiguous in memory.

Figure: memory layout diagram (group edges, block structure). Reuse for talk.

3.2 Kernel design

Draft from repo design notes `docs/kernels.md`. Keep to what a reader needs to reproduce performance; full design-pattern discussion goes to the thesis chapter.

The Woodbury inner matrix $\left(\mathbf{I} + \mathbf{\Delta}^\dagger \mathbf{N}^{-1} \mathbf{\Delta}\right)$ is block diagonal over groups, so its Cholesky factorisation is performed as a batched operation with one block per group.

3.3 Numerical precision

All accumulations are performed in double precision.

State why: FP32 accumulation failed, see attic A1. One sentence here, the detail stays in the attic/thesis.

3.4 Validation

GPU outputs are compared against the reference CPU implementation for the likelihood, gradient, and final gains on simulated arrays. [results.md 2026-10-14]

4 Results

Claim: The GPU implementation matches the CPU reference to machine precision and is [N]× faster at CHORD scale; on quasi-redundant simulations it recovers gains with lower bias than redundant calibration.

4.1 Agreement with the reference implementation

FIGURE PLACEHOLDER

Fractional difference between GPU and CPU χ^2 and gradient vs. array size

Status: script exists: `scripts/fig_validation.py`*; rerun with final kernels*

Figure 1: Fractional difference between GPU and CPU χ^2 and gradient vs. array size

Across [range of array sizes], the fractional difference in χ^2 is below [value]. [results.md 2026-10-14]

4.2 Performance scaling

FIGURE PLACEHOLDER

Wall time per likelihood+gradient evaluation vs. number of antennas, CPU and GPU

Status: not yet run at full CHORD size

Figure 2: Wall time per likelihood+gradient evaluation vs. number of antennas, CPU and GPU

Decide hardware to report (cluster GPU model) and fix the benchmark protocol before running final numbers.

4.3 Calibration accuracy under non-redundancy

Earlier version claimed CorrCal also improves convergence speed. Not supported, moved to attic A3. Do not reintroduce without new evidence.

FIGURE PLACEHOLDER

Gain bias vs. antenna position scatter, CorrCal vs. redundant calibration

Status: simulation pipeline in progress

Figure 3: Gain bias vs. antenna position scatter, CorrCal vs. redundant calibration

5 Discussion and outlook

Claim: GPU CorrCal makes covariance-based calibration routine for CHORD and is the foundation for extensions to the sky model.

Fill once results are final. Candidate points: limits of the low-rank diffuse model; application to real CHORD data; extensions (time and frequency correlated gains).

References

[1] R. Pascua et al. [title placeholder: export from zotero]. 2026.

[2] J. Sievers. Calibration of quasi-redundant interferometers. *arXiv e-prints*, 2017.